

DOCUMENTO DE DESARROLLO DEL PROYECTO

Sistema Inteligente de Gestión y Análisis Documental – DocuMind

Programa: Tecnología en Desarrollo de Software

Nombre: Reyner Santiago Acosta Pinto

Semestre: VI

Asignatura: Desarrollo de Aplicaciones Empresariales

Institución: Unidades Tecnológicas de Santander – UTS

Docente: Wilson Castaño Galviz

Proyecto: Sistema Inteligente de Gestión y Análisis Documental

Nombre del sistema: DocuMind

CONTROL DEL DOCUMENTO

Campo	Información
Proyecto	DocuMind – Sistema Inteligente de Gestión y Análisis Documental
Documento	02 – Documento de Diseño
Fase	Diseño
Versión	1.0
Institución	Unidades Tecnológicas de Santander – UTS
Programa	Tecnología en Desarrollo de Software
Asignatura	Desarrollo de Aplicaciones Empresariales
Docente	Wilson Castaño Galviz
Repositorio	DesarrolloE2
Estado	Documento de diseño

1. INTRODUCCIÓN

El presente documento corresponde a la fase de **Diseño** del proyecto integrador **DocuMind – Sistema Inteligente de Gestión y Análisis Documental**.

El objetivo de esta fase es transformar los requisitos definidos durante el análisis en una propuesta técnica que establezca cómo estará estructurado el sistema, cómo se comunicarán sus componentes, cómo se almacenará la información, cómo se integrará el procesamiento documental con la Inteligencia Artificial y cómo se garantizarán aspectos básicos de seguridad, mantenibilidad y operación.

DocuMind es una aplicación web orientada a la gestión de documentos empresariales en formatos PDF, DOCX y TXT. El sistema incorpora mecanismos de extracción de contenido, clasificación, generación de resúmenes, extracción de información relevante, búsqueda documental y consultas en lenguaje natural.

La solución implementada utiliza **PHP, HTML, CSS, JavaScript, MySQL/MariaDB, Apache y un servicio de Inteligencia Artificial compatible con API**, contemplando además la posibilidad de utilizar Ollama para procesamiento local.

El diseño se fundamenta en la implementación existente del proyecto y en el esquema de base de datos disponible en el repositorio. La base de datos contiene las entidades users, repositories, documents y processing_logs, mientras que el procesador documental concentra las funciones de extracción, clasificación y análisis.

2. OBJETIVOS DEL DISEÑO

2.1 Objetivo general

Definir la arquitectura, estructura de datos, componentes, interfaces, flujos de comunicación, integración de Inteligencia Artificial y mecanismos básicos de seguridad necesarios para implementar y mantener el sistema DocuMind de acuerdo con los requisitos establecidos en la fase de análisis.

2.2 Objetivos específicos

1. Definir la arquitectura lógica y física del sistema.
2. Identificar los principales componentes de la aplicación.
3. Establecer la comunicación entre frontend, backend, base de datos, almacenamiento documental y servicio de IA.
4. Diseñar la estructura relacional de la base de datos.
5. Definir las entidades, atributos, claves y relaciones.
6. Diseñar el flujo de procesamiento de documentos.

7. Diseñar la integración entre el procesador documental y el servicio de Inteligencia Artificial.
 8. Definir las interfaces principales de usuario.
 9. Establecer los flujos de navegación.
 10. Definir mecanismos básicos de seguridad.
 11. Establecer los diagramas técnicos que deberán acompañar la documentación.
 12. Mantener correspondencia entre el diseño y la implementación real del proyecto.
-

3. ARQUITECTURA GENERAL DEL SISTEMA

3.1 Descripción

DocuMind utiliza una arquitectura web cliente-servidor.

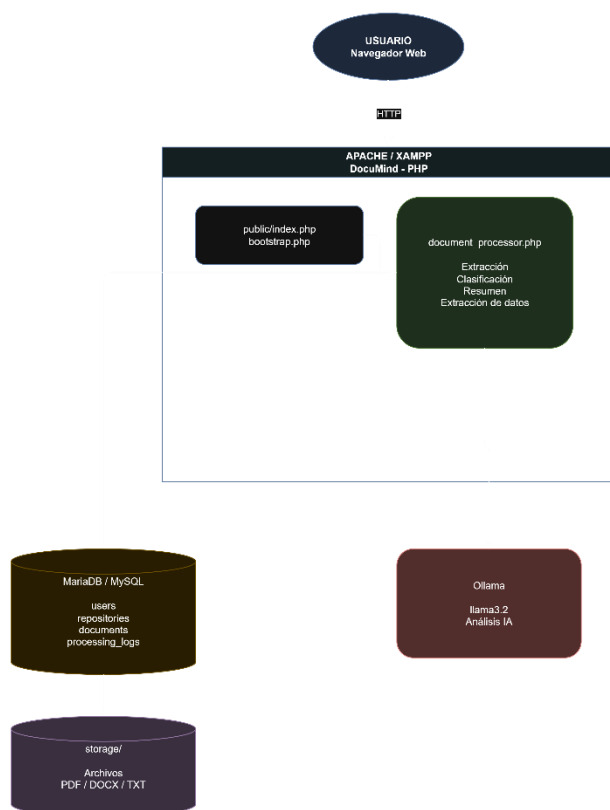
El usuario interactúa con el sistema desde un navegador web. Las solicitudes llegan al servidor Apache, donde PHP procesa las operaciones, aplica las reglas correspondientes, consulta la base de datos y devuelve las vistas HTML.

Cuando se realiza una operación documental, el backend puede utilizar el procesador documental para extraer contenido y realizar el análisis correspondiente.

La arquitectura general puede dividirse en los siguientes componentes:

1. Cliente web.
 2. Servidor web.
 3. Aplicación PHP.
 4. Procesador documental.
 5. Base de datos MySQL/MariaDB.
 6. Almacenamiento físico de archivos.
 7. Servicio de Inteligencia Artificial.
-

4. DIAGRAMA DE ARQUITECTURA



Descripción del diagrama

El usuario accede al sistema mediante un navegador web. Las solicitudes HTTP son recibidas por Apache y procesadas por PHP.

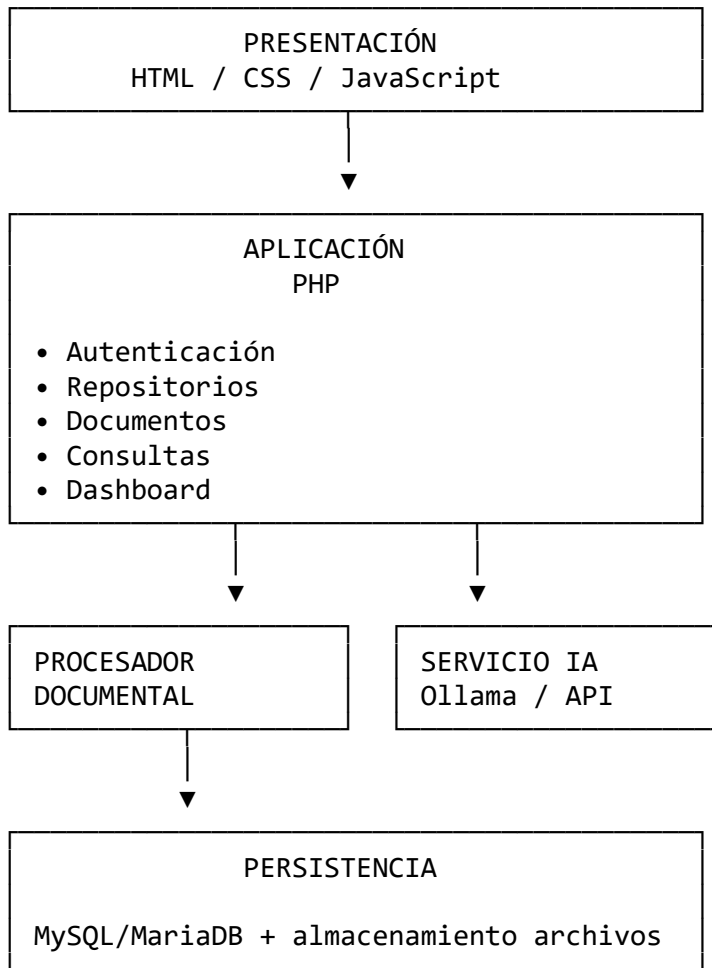
La aplicación utiliza una base de datos MySQL/MariaDB para almacenar usuarios, repositorios, documentos, resultados del procesamiento y registros de procesamiento.

Los archivos originales son almacenados en el sistema de archivos, mientras que la información descriptiva y el contenido extraído se almacenan en la base de datos.

Cuando un documento requiere análisis, el backend utiliza el componente de procesamiento documental. Este componente puede realizar extracción local y comunicarse con el servicio de Inteligencia Artificial configurado.

5. ARQUITECTURA LÓGICA

La arquitectura lógica del sistema puede organizarse en los siguientes niveles:



6. COMPONENTES DEL SISTEMA

6.1 Cliente web

El cliente corresponde al navegador utilizado por el usuario.

Su responsabilidad es:

- Mostrar las interfaces.
 - Capturar formularios.
 - Enviar solicitudes HTTP.
 - Mostrar mensajes.
 - Presentar resultados.
 - Permitir interacción con el dashboard.
-

6.2 Aplicación PHP

Es el núcleo de la aplicación.

Se encarga de:

- Inicializar el sistema.
- Gestionar sesiones.
- Validar autenticación.
- Procesar solicitudes.
- Consultar la base de datos.
- Gestionar repositorios.
- Gestionar documentos.
- Invocar el procesamiento documental.
- Generar las respuestas HTML.

El punto de entrada actual requiere `bootstrap.php` y `document_processor.php`, lo que demuestra la separación entre inicialización de la aplicación y procesamiento documental.

6.3 Bootstrap

El componente `bootstrap.php` centraliza aspectos comunes de la aplicación.

Entre sus responsabilidades se encuentran:

- Configuración.
- Conexión a la base de datos.
- Sesiones.
- Autenticación.
- Protección CSRF.
- Funciones comunes.

Esto permite evitar duplicación de lógica entre las diferentes operaciones.

6.4 Procesador documental

El archivo:

`app/document_processor.php`

contiene la lógica relacionada con el procesamiento documental.

Entre sus responsabilidades se encuentran:

- Extracción de TXT.
- Extracción de DOCX.
- Extracción de PDF.
- Clasificación.
- Identificación del tipo documental.
- Extracción de información.
- Generación de resumen.
- Comunicación con IA.

El código actual implementa extracción de TXT mediante lectura directa, DOCX mediante ZipArchive y PDF mediante procesamiento de sus streams.

6.5 Base de datos

La base de datos utilizada es **MySQL/MariaDB**.

Su función es almacenar:

- Usuarios.
- Repositorios.
- Documentos.
- Contenido extraído.
- Categorías.
- Resúmenes.
- Datos extraídos.
- Estados.
- Errores.
- Logs.

El esquema actual utiliza utf8mb4 y relaciones mediante claves foráneas.

6.6 Almacenamiento de archivos

Los documentos originales no se almacenan directamente dentro de la base de datos como archivos binarios.

La tabla documents conserva:

- Nombre original.

- Nombre almacenado.
- MIME.
- Extensión.
- Tamaño.

El archivo físico se conserva en el directorio de almacenamiento configurado por la aplicación.

6.7 Servicio de Inteligencia Artificial

La aplicación puede comunicarse con un servicio compatible con una API de modelos de lenguaje.

La configuración contempla procesamiento local mediante **Ollama**, permitiendo que el análisis pueda ejecutarse sin depender necesariamente de un proveedor externo.

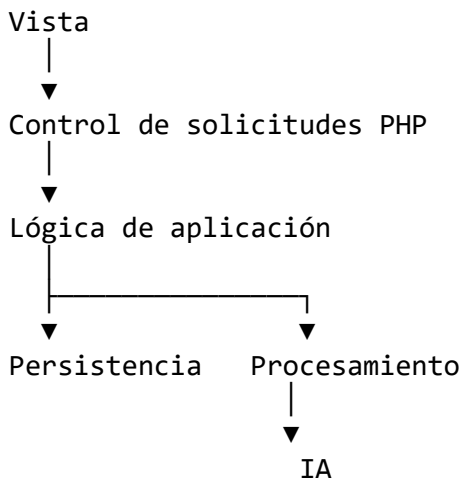
7. MODELO ARQUITECTÓNICO

7.1 Patrón utilizado

DocuMind utiliza una organización por responsabilidades que puede describirse como una arquitectura web por capas.

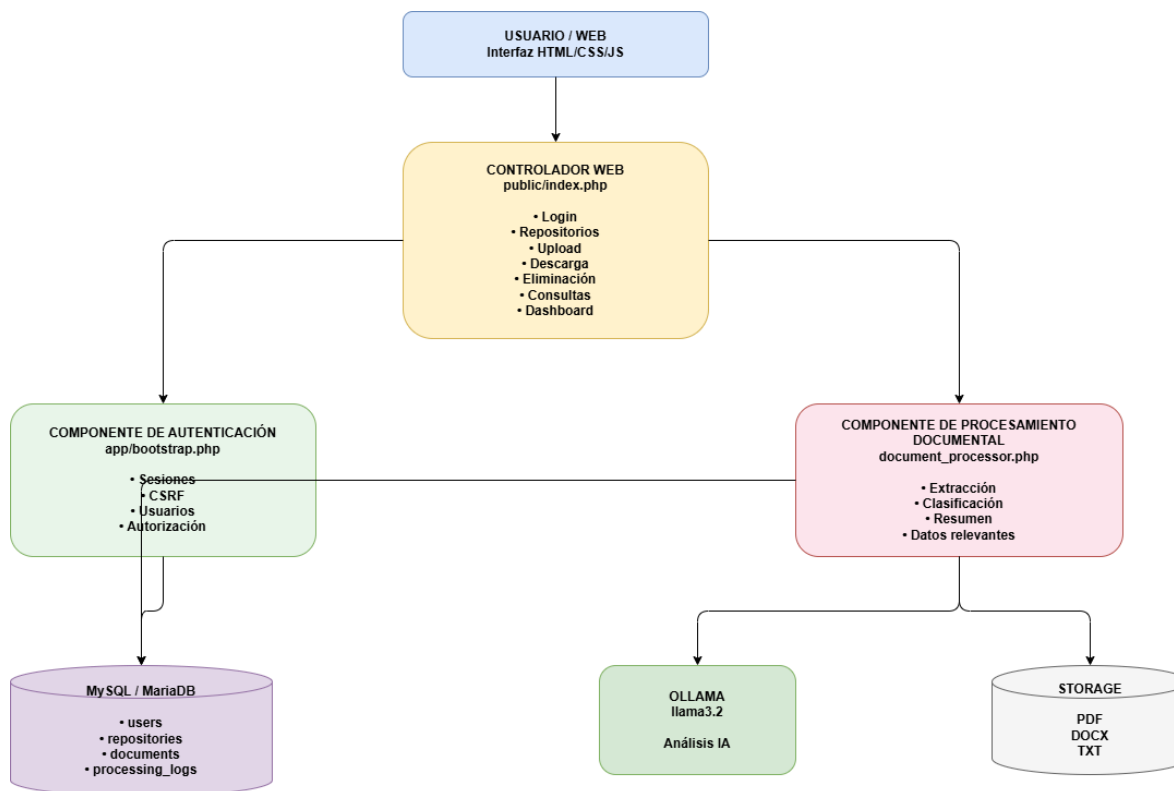
No se debe presentar el sistema como un MVC clásico de frameworks si la implementación no utiliza un framework MVC formal.

La estructura real se aproxima conceptualmente a:

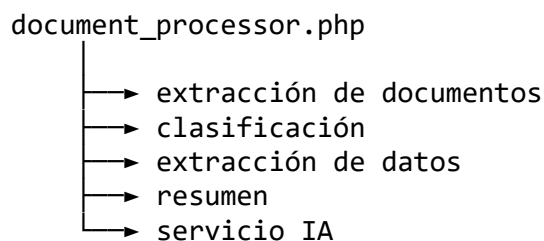
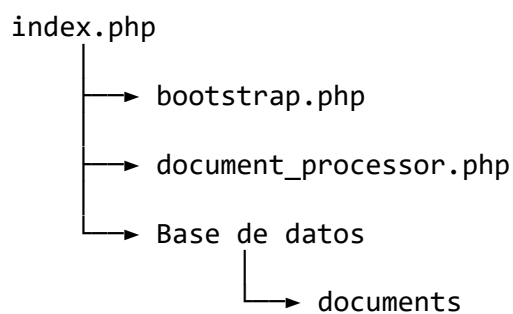


Esta descripción es más fiel al código actual que afirmar que se utiliza un framework MVC completo.

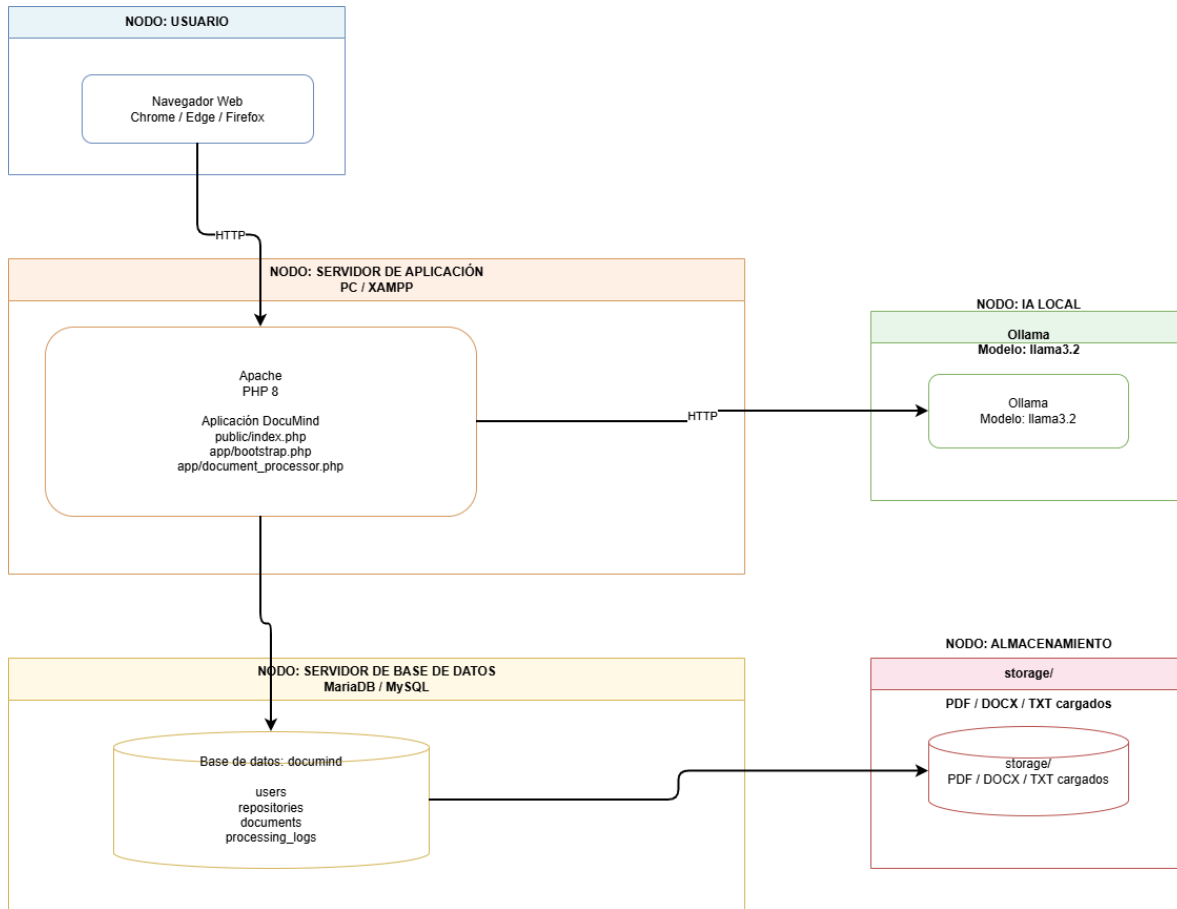
8. DIAGRAMA DE COMPONENTES



Relaciones principales



9. DIAGRAMA DE DESPLIEGUE



Nodo cliente:

- Computador.
- Navegador web.

Nodo servidor:

- Apache.
- PHP.
- Aplicación DocuMind.

Nodo base de datos:

- MySQL/MariaDB.

Nodo almacenamiento:

- Directorio de documentos.

Nodo IA:

- Ollama local o servicio compatible con API.

Descripción

El usuario utiliza un computador para acceder a DocuMind mediante un navegador.

Apache recibe las solicitudes y ejecuta PHP.

PHP se comunica con MySQL/MariaDB para obtener y almacenar información.

Los archivos originales son almacenados en el sistema de archivos.

Cuando el procesamiento IA está habilitado, la aplicación se comunica con Ollama o con el servicio compatible configurado.

10. MODELO DE DATOS

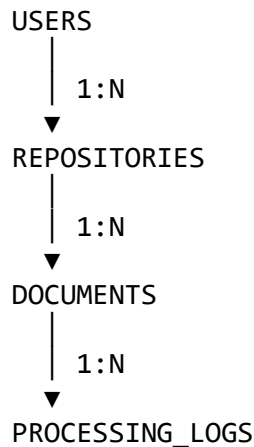
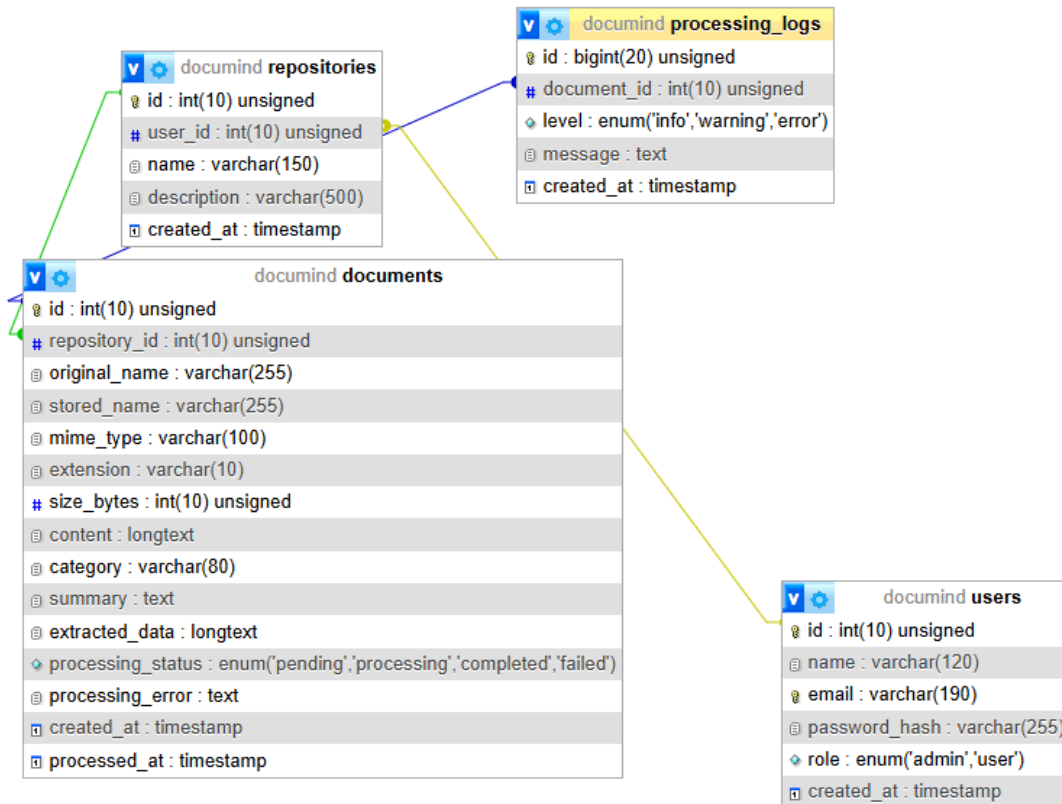
10.1 Descripción general

El sistema utiliza un modelo relacional compuesto principalmente por cuatro entidades:

1. users
2. repositories
3. documents
4. processing_logs

La estructura actual define claves primarias, claves foráneas, restricciones y eliminación en cascada.

11. MODELO ENTIDAD-RELACIÓN



Explicación

Un usuario puede tener múltiples repositorios.

Un repositorio puede contener múltiples documentos.

Un documento puede tener múltiples registros de procesamiento.

La relación users → repositories utiliza user_id.

La relación repositories → documents utiliza repository_id.

La relación documents → processing_logs utiliza document_id.

Las relaciones principales están definidas mediante claves foráneas en el esquema SQL.

12. DISEÑO DE BASE DE DATOS

12.1 Tabla users

Almacena los usuarios del sistema.

Campo	Tipo	Restricción	Descripción
id	INT UNSIGNED	PK, AI	Identificador
name	VARCHAR(120)	NOT NULL	Nombre
email	VARCHAR(190)	UNIQUE, NOT NULL	Correo
password_hash	VARCHAR(255)	NOT NULL	Contraseña cifrada/hash
role	ENUM	NOT NULL	Rol
created_at	TIMESTAMP	NOT NULL	Fecha de creación

El esquema actual contempla los roles admin y user.

13. TABLA repositories

Almacena los repositorios creados por los usuarios.

Campo	Tipo	Restricción	Descripción
id	INT UNSIGNED	PK, AI	Identificador
user_id	INT UNSIGNED	FK	Usuario propietario
name	VARCHAR(150)	NOT NULL	Nombre
description	VARCHAR(500)	NULL	Descripción
created_at	TIMESTAMP	NOT NULL	Fecha de creación

La clave foránea user_id referencia a users(id) y utiliza eliminación en cascada.

14. TABLA documents

Es la entidad central del sistema documental.

Campo	Tipo	Restricción	Descripción
id	INT UNSIGNED	PK, AI	Identificador
repository_id	INT UNSIGNED	FK	Repositorio
original_name	VARCHAR(255)	NOT NULL	Nombre original
stored_name	VARCHAR(255)	NOT NULL	Nombre interno
mime_type	VARCHAR(100)	NOT NULL	Tipo MIME
extension	VARCHAR(10)	NOT NULL	Extensión
size_bytes	INT UNSIGNED	NOT NULL	Tamaño
content	LONGTEXT	NULL	Texto extraído
category	VARCHAR(80)	NULL	Categoría
summary	TEXT	NULL	Resumen
extracted_data	JSON	NULL	Datos extraídos
processing_status	ENUM	NOT NULL	Estado
processing_error	TEXT	NULL	Error
created_at	TIMESTAMP	NOT NULL	Fecha de carga
processed_at	TIMESTAMP	NULL	Fecha de procesamiento

La tabla también posee un índice FULLTEXT sobre original_name, content y summary, destinado a facilitar las búsquedas documentales.

15. TABLA processing_logs

Registra eventos relacionados con el procesamiento.

Campo	Tipo	Restricción	Descripción
id	BIGINT UNSIGNED	PK, AI	Identificador
document_id	INT UNSIGNED	FK, NULL	Documento
level	ENUM	NOT NULL	Nivel
message	TEXT	NOT NULL	Mensaje
created_at	TIMESTAMP	NOT NULL	Fecha

Los niveles contemplados son:

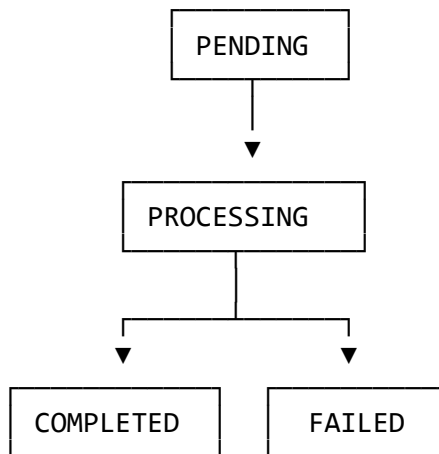
- info

- warning
- error

La relación utiliza `document_id` y permite establecerlo como NULL cuando el documento relacionado es eliminado.

16. ESTADOS DEL PROCESAMIENTO

La entidad `documents` utiliza cuatro estados:



`pending`

Documento registrado pero pendiente de procesamiento.

`processing`

Documento actualmente en proceso.

`completed`

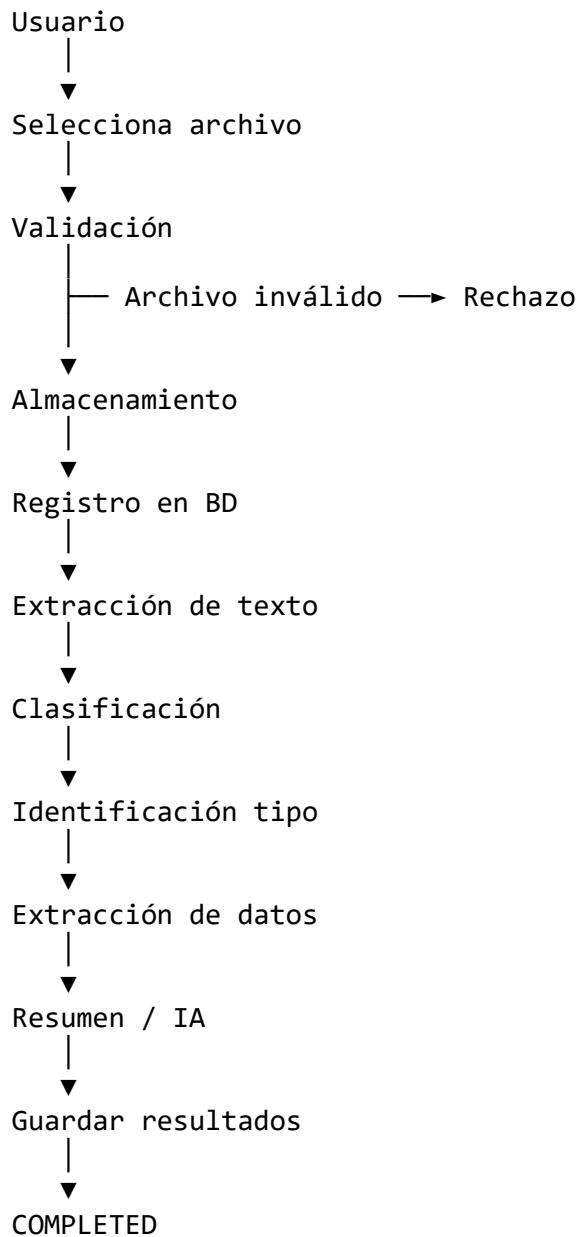
Procesamiento terminado correctamente.

`failed`

Se produjo un error durante el procesamiento.

17. DISEÑO DEL FLUJO DOCUMENTAL

17.1 Flujo general



18. EXTRACCIÓN DE TEXTO

El sistema contempla diferentes mecanismos dependiendo de la extensión.

18.1 TXT

Se realiza una lectura directa del archivo.

18.2 DOCX

El documento DOCX se trata como un archivo comprimido y se obtiene el contenido de:

`word/document.xml`

Posteriormente se eliminan las etiquetas XML y se reconstruye el texto.

18.3 PDF

El procesador inspecciona los streams internos del PDF, intenta descomprimirlos y busca fragmentos textuales.

El contenido resultante se normaliza y se limita a 12.000 caracteres en la función de extracción actual.

Consideración de diseño

Este mecanismo es apropiado para la versión académica actual, pero debe documentarse como una limitación: PDFs escaneados o estructuras PDF complejas pueden requerir OCR o bibliotecas especializadas.

19. DISEÑO DE CLASIFICACIÓN

La clasificación documental se encuentra diseñada para trabajar con cuatro categorías:

Categoría	Palabras de referencia
Finanzas	factura, presupuesto, pago, contabilidad, financiero, proveedor
Recursos humanos	empleado, contrato, nómina, vacaciones, talento
Operaciones	proceso, inventario, logística, informe, procedimiento
General	Cuando no existe coincidencia suficiente

El procesador actual combina el texto y el nombre del archivo para realizar la clasificación basada en reglas.

20. DISEÑO DE IDENTIFICACIÓN DOCUMENTAL

El sistema puede identificar diferentes tipos documentales.

Tipo	Ejemplos
Factura	Facturas, comprobantes de compra
Contrato	Contratos laborales o comerciales
Informe	Informes administrativos u operativos
General	Documentos que no encajan en los tipos anteriores

Esta identificación sirve como contexto para la extracción de información relevante.

21. DISEÑO DE EXTRACCIÓN DE INFORMACIÓN

El componente documental puede obtener información mediante patrones y reglas.

Entre los datos considerados se encuentran:

- Correos electrónicos.
- Fechas.
- Valores monetarios.
- Número de factura.
- Proveedor.
- Partes de contratos.
- Responsable.
- Conclusiones.

Los resultados se almacenan en el campo `extracted_data`, definido como tipo JSON.

22. INTEGRACIÓN CON INTELIGENCIA ARTIFICIAL

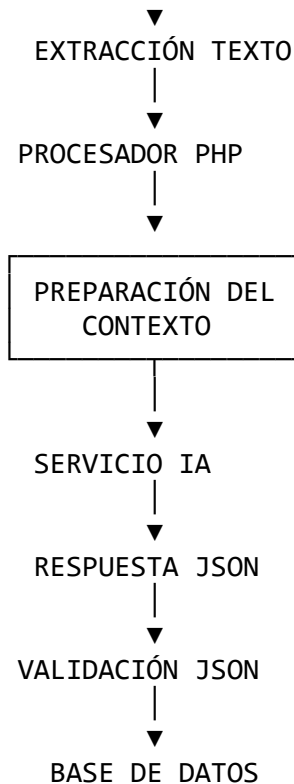
22.1 Objetivo

La IA complementa el procesamiento documental permitiendo generar información estructurada y responder consultas sobre los documentos.

22.2 Arquitectura de integración

DOCUMENTO

|



23. SERVICIO DE IA

El diseño permite trabajar con un servicio local basado en **Ollama**.

La arquitectura contempla una comunicación HTTP desde PHP hacia el servicio de IA.

La ventaja de esta alternativa es que permite realizar el procesamiento localmente, reduciendo la dependencia de servicios externos y evitando enviar necesariamente documentos empresariales a terceros.

El repositorio documenta la posibilidad de utilizar Ollama con un modelo configurado localmente.

24. FORMATO DE RESPUESTA DE IA

El diseño contempla una respuesta estructurada en JSON.

Conceptualmente:

```
{  
  "category": "Finanzas",
```

```
    "document_type": "Factura",
    "summary": "Resumen del documento...",
    "relevant_data": {
        "emails": [],
        "dates": [],
        "amounts": []
    }
}
```

La estructura permite que PHP interprete la respuesta de manera determinista.

La aplicación valida que la respuesta pueda ser interpretada como JSON y que contenga información esperada antes de almacenar los resultados.

25. DISEÑO DE CONSULTAS DOCUMENTALES

La consulta documental se plantea en dos etapas:

Etapas 1 – Recuperación

El sistema busca documentos relacionados con los términos proporcionados por el usuario.

La base de datos dispone de un índice FULLTEXT sobre:

- original_name
- content
- summary

lo que proporciona una estructura apropiada para realizar búsquedas sobre el contenido documental.

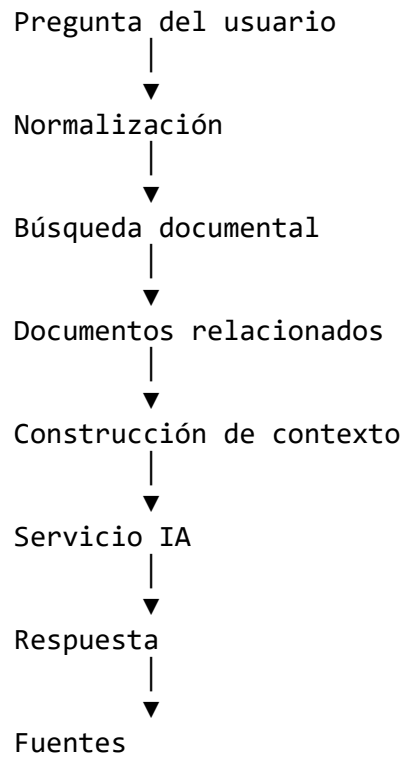
Etapas 2 – Respuesta

Los documentos recuperados proporcionan el contexto utilizado para responder la pregunta.

La respuesta deberá estar relacionada con los documentos encontrados.

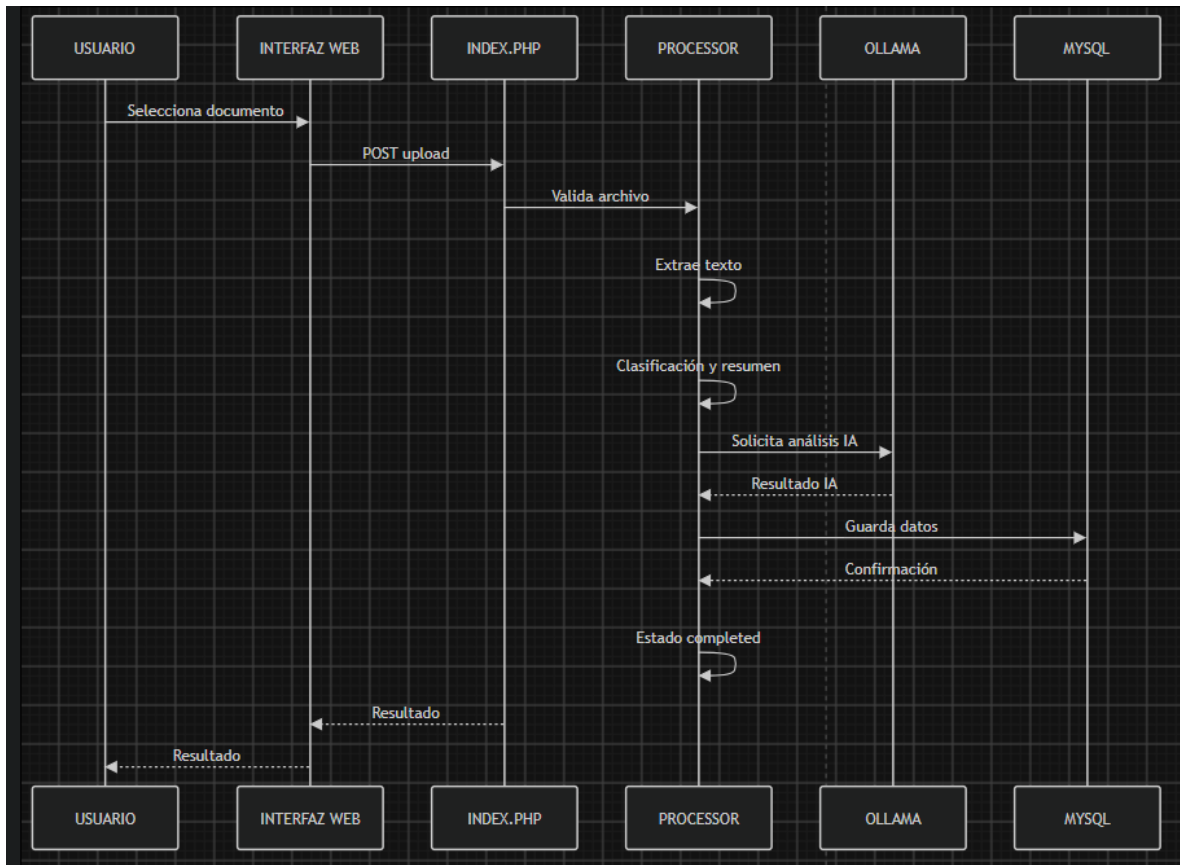
26. DISEÑO DE CONSULTAS EN LENGUAJE NATURAL

El flujo conceptual es:



Si no existen documentos relacionados, el sistema deberá informar que no se encontró información suficiente.

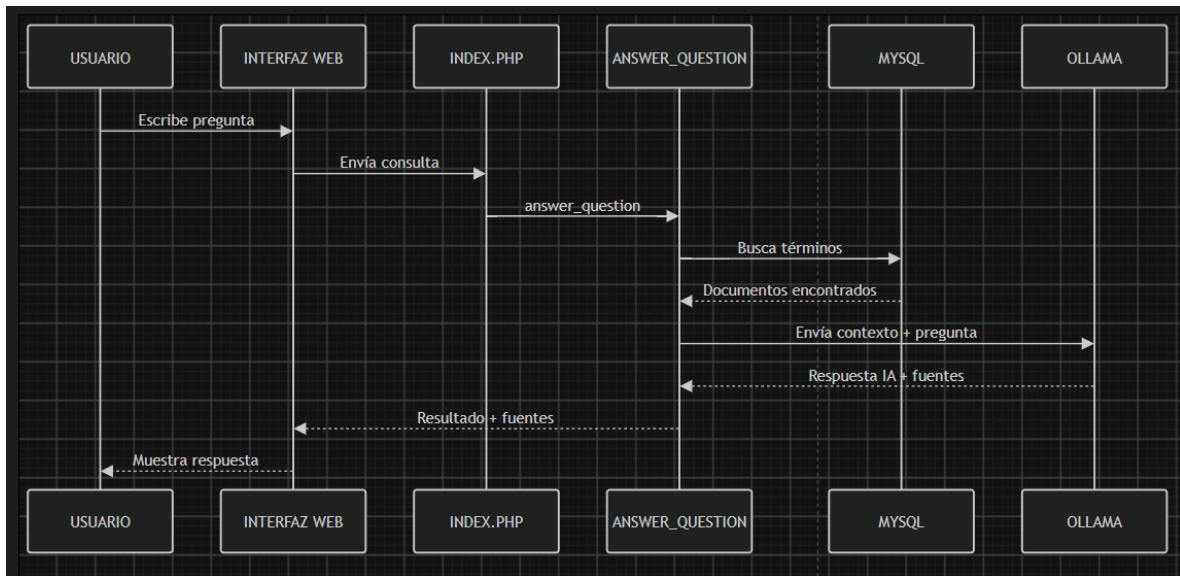
27. DIAGRAMA DE SECUENCIA – CARGA Y PROCESAMIENTO



Secuencia

1. Usuario selecciona archivo.
2. Navegador envía solicitud.
3. PHP valida sesión.
4. PHP valida archivo.
5. PHP almacena archivo.
6. PHP crea registro.
7. Procesador extrae contenido.
8. Procesador realiza análisis.
9. Procesador solicita IA.
10. IA devuelve resultado.
11. PHP valida resultado.
12. PHP almacena información.
13. PHP actualiza estado.
14. Sistema informa resultado.

28. DIAGRAMA DE SECUENCIA – CONSULTA



Participantes

- Usuario.
- Navegador.
- Aplicación PHP.
- MySQL/MariaDB.
- Servicio IA.

Secuencia

1. Usuario introduce pregunta.
2. Aplicación recibe la solicitud.
3. Se valida la sesión.
4. Se buscan documentos relacionados.
5. Se recuperan contenidos relevantes.
6. Se construye el contexto.
7. Se consulta la IA.
8. Se recibe la respuesta.
9. Se identifican las fuentes.
10. Se muestra el resultado.

29. SECUENCIA – AUTENTICACIÓN

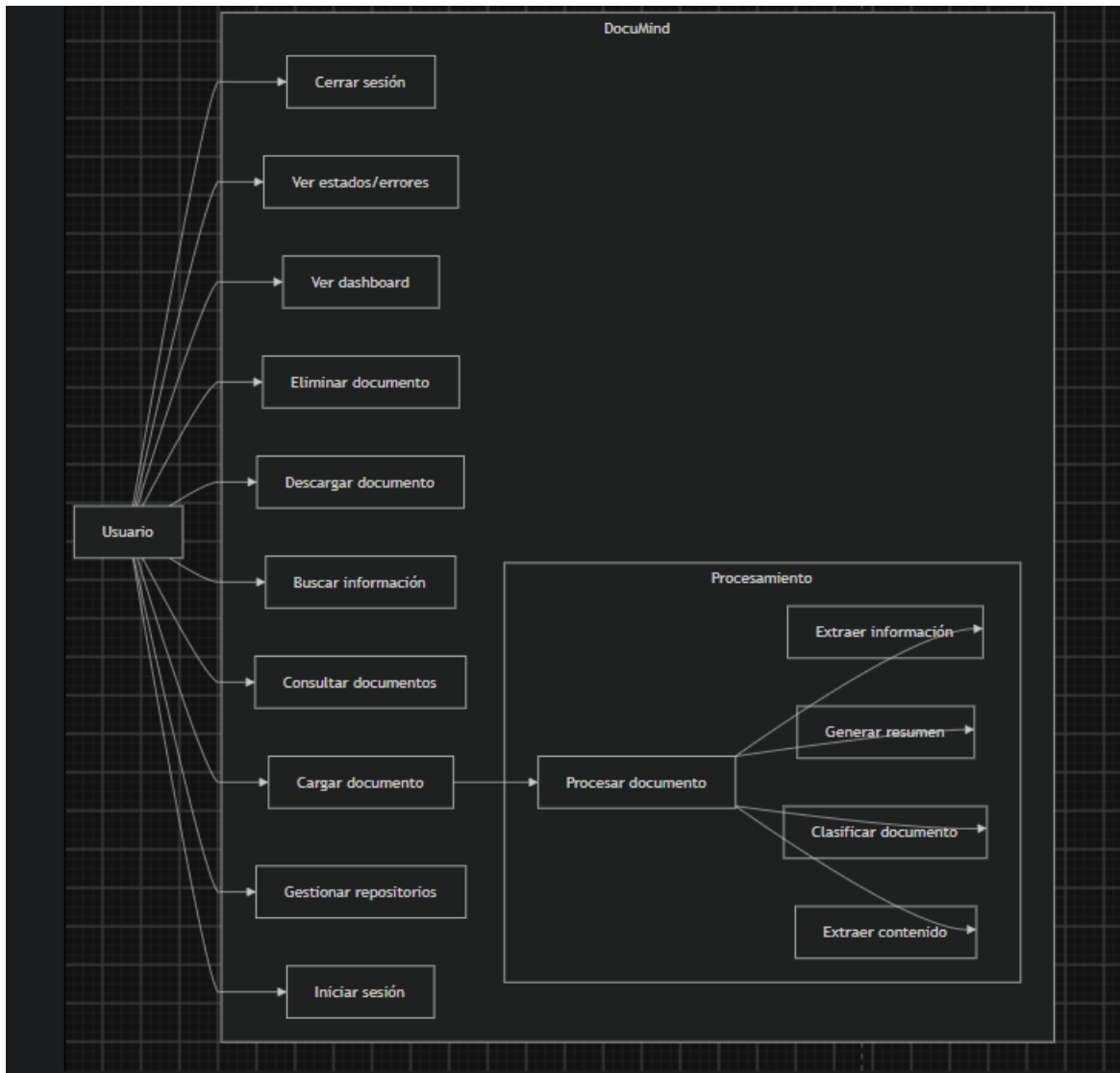
Flujo

1. Usuario introduce correo y contraseña.

2. Navegador envía solicitud.
3. PHP valida CSRF.
4. PHP consulta usuario.
5. Base de datos devuelve información.
6. PHP verifica contraseña.
7. Se regenera el identificador de sesión.
8. Se guarda el usuario en sesión.
9. Se redirige al dashboard.

El código actual utiliza `password_verify()` y regenera el identificador de sesión después de una autenticación correcta.

30. DIAGRAMA DE CASOS DE USO



Actores

- Usuario.
- Administrador.
- Servicio de IA.

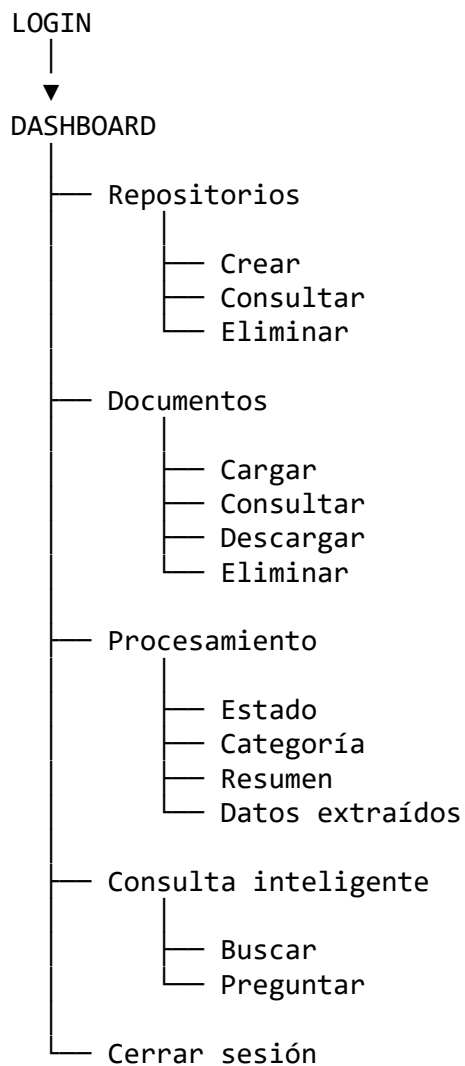
Casos principales

- Iniciar sesión.
- Cerrar sesión.
- Crear repositorio.
- Consultar repositorios.
- Cargar documento.

- Procesar documento.
 - Consultar documento.
 - Buscar información.
 - Preguntar sobre documentos.
 - Descargar documento.
 - Eliminar documento.
 - Eliminar repositorio.
 - Consultar dashboard.
 - Consultar estado de procesamiento.
-

31. DISEÑO DE NAVEGACIÓN

El sistema se organiza alrededor del dashboard.



33. DISEÑO DE INTERFAZ – LOGIN

Objetivo

Permitir la autenticación del usuario.

Elementos

- Nombre/logo DocuMind.
- Campo correo.
- Campo contraseña.
- Botón ingresar.
- Mensaje de error.

Validaciones

- Correo obligatorio.
 - Contraseña obligatoria.
 - Credenciales válidas.
 - Protección CSRF.
-
-

35. DISEÑO DE INTERFAZ – DASHBOARD

El dashboard constituye la pantalla principal del usuario autenticado.

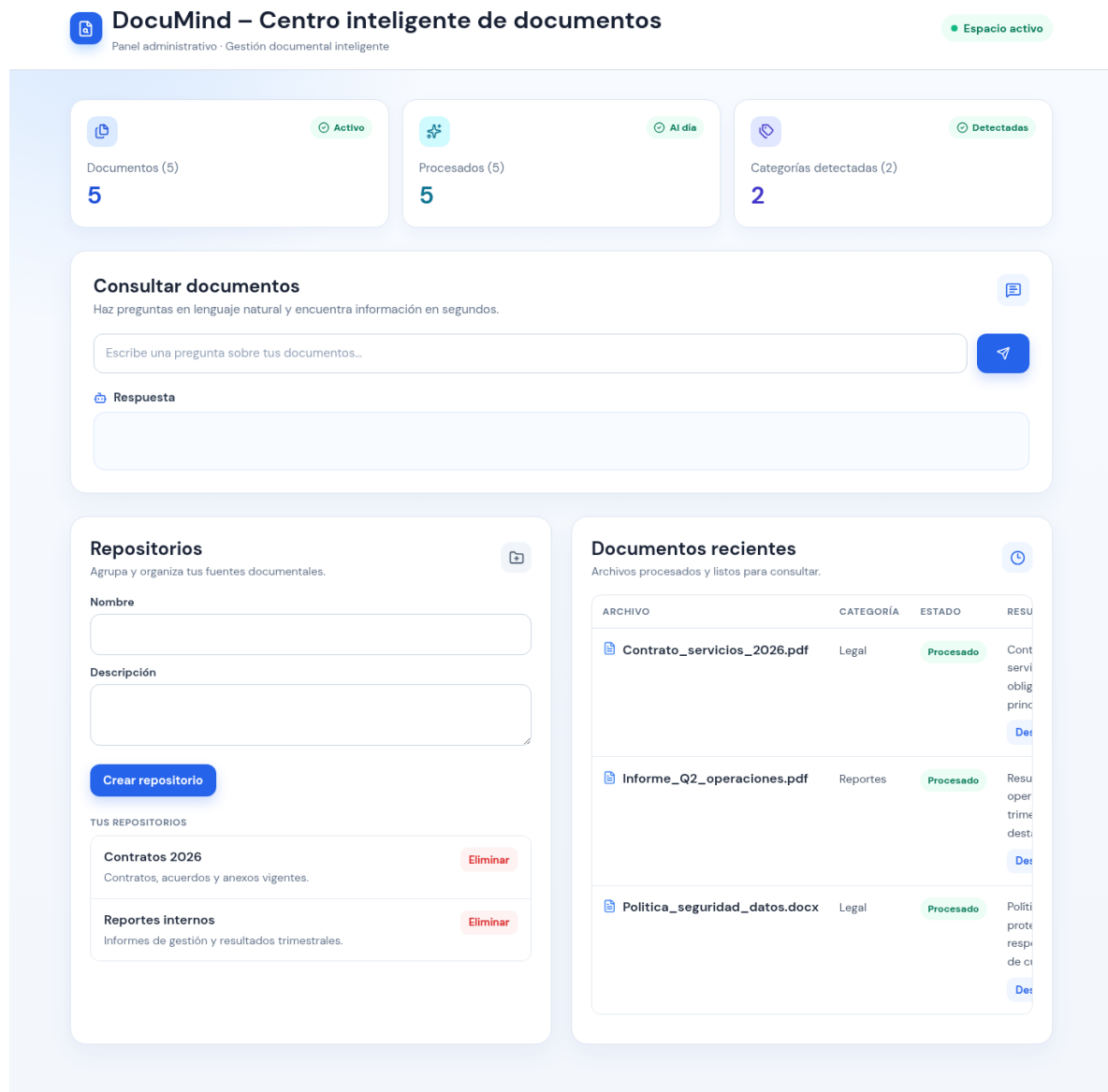
Elementos principales

- Nombre del sistema.
- Información del usuario.
- Indicadores.
- Repositorios.
- Documentos recientes.
- Estado de procesamiento.
- Acciones documentales.
- Módulo de consultas.

Indicadores

- Documentos totales.
- Documentos procesados.
- Categorías.
- Estados.

36. MOCKUP DASHBOARD



37. DISEÑO DE INTERFAZ – CARGA DE DOCUMENTOS

Elementos

- Selector de repositorio.
- Selector de archivo.
- Botón de carga.
- Mensaje de validación.
- Estado del procesamiento.

Validaciones

- Extensión permitida.
 - Tamaño máximo.
 - Repositorio válido.
 - Usuario autenticado.
-
-

39. DISEÑO DE INTERFAZ – INFORMACIÓN DEL DOCUMENTO

La interfaz del documento deberá permitir visualizar información obtenida durante el procesamiento.

Información

- Nombre original.
- Tipo MIME.
- Extensión.
- Tamaño.
- Categoría.
- Tipo documental.
- Resumen.
- Datos relevantes.
- Estado.
- Fecha de procesamiento.

Acciones

- Descargar.

- Eliminar.
 - Consultar.
-
-

41. DISEÑO DE INTERFAZ – CONSULTA INTELIGENTE

Elementos

- Campo para escribir pregunta.
- Botón consultar.
- Área de respuesta.
- Lista de fuentes.

Ejemplo

Pregunta:

¿Cuál es el valor total de las facturas encontradas?

Respuesta:

El sistema encontró información relacionada en los documentos recuperados.

Fuentes:

- factura_enero.pdf
 - factura_febrero.pdf
-
-

43. DISEÑO DE COMPONENTES DE SEGURIDAD

La seguridad se considera transversal a la aplicación.

43.1 Autenticación

El usuario debe autenticarse antes de acceder a las funcionalidades protegidas.

43.2 Contraseñas

Las contraseñas no deberán almacenarse directamente.

La tabla utiliza el campo password_hash.

43.3 Sesiones

La aplicación utiliza sesiones PHP para mantener el usuario autenticado.

Después de iniciar sesión correctamente se regenera el identificador de sesión.

43.4 CSRF

Las operaciones sensibles utilizan tokens CSRF.

43.5 SQL Injection

Las consultas deben utilizar PDO y parámetros preparados.

43.6 Control de propiedad

Las operaciones documentales deben verificar que el documento pertenece a un repositorio del usuario autenticado.

Por ejemplo, la descarga actual consulta simultáneamente el documento y el `user_id` del repositorio antes de permitir la entrega del archivo.

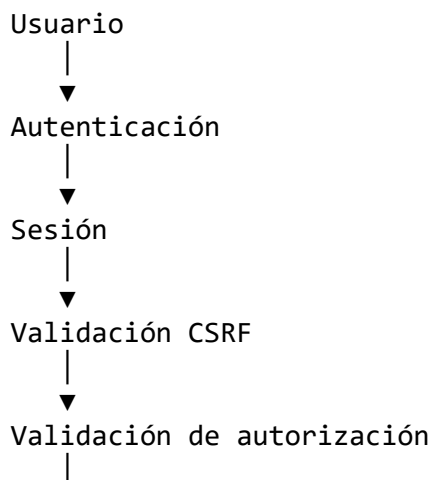
43.7 Archivos

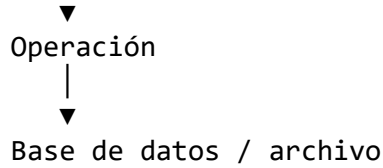
Los archivos deben almacenarse utilizando nombres internos diferentes al nombre proporcionado originalmente por el usuario.

Esto evita depender directamente del nombre original para construir rutas físicas.

44. DISEÑO DE SEGURIDAD

44.1 Flujo





45. DISEÑO DE MANEJO DE ERRORES

El sistema debe manejar errores en diferentes niveles.

Nivel 1 – Validación

Ejemplos:

- Archivo no permitido.
- Archivo demasiado grande.
- Repositorio inexistente.
- Datos incompletos.

Nivel 2 – Procesamiento

Ejemplos:

- Error de extracción.
- Documento sin contenido.
- Error de clasificación.
- Error de procesamiento.

Nivel 3 – IA

Ejemplos:

- Servicio no disponible.
- Timeout.
- Respuesta inválida.
- JSON incorrecto.

Nivel 4 – Persistencia

Ejemplos:

- Error de conexión.
- Error SQL.
- Restricción de integridad.

Los errores importantes del procesamiento se registran mediante `processing_logs`.

46. DISEÑO DE LOGS

Los logs se almacenan en la tabla:

`processing_logs`

Niveles

Nivel	Uso
info	Evento normal
warning	Situación anormal no crítica
error	Fallo del procesamiento

Ejemplo

INFO

Documento cargado correctamente.

INFO

Extracción de contenido completada.

INFO

Documento procesado.

ERROR

El servicio de IA no respondió.

ERROR

No fue posible extraer contenido.

47. DECISIONES TECNOLÓGICAS

47.1 PHP

Se utiliza PHP porque permite desarrollar el backend web de manera sencilla y compatible con el entorno académico basado en XAMPP.

47.2 Apache

Apache proporciona el servidor web necesario para ejecutar PHP.

47.3 MySQL/MariaDB

Se selecciona una base de datos relacional porque la información posee relaciones claras entre usuarios, repositorios, documentos y logs.

47.4 HTML/CSS/JavaScript

Permiten construir la interfaz web sin requerir un proceso de compilación frontend complejo.

47.5 Ollama

Se contempla Ollama como alternativa para ejecutar modelos de IA localmente.

47.6 PHP PDO

PDO proporciona una interfaz adecuada para trabajar con MySQL/MariaDB y permite utilizar consultas preparadas.

48. JUSTIFICACIÓN DE LA ARQUITECTURA

La arquitectura seleccionada responde a las características del proyecto:

Simplicidad

El sistema puede ejecutarse en un entorno local sin infraestructura compleja.

Mantenibilidad

La separación entre configuración, aplicación y procesamiento facilita modificar componentes.

Reproducibilidad

El proyecto puede reconstruirse mediante el código fuente y el script de base de datos.

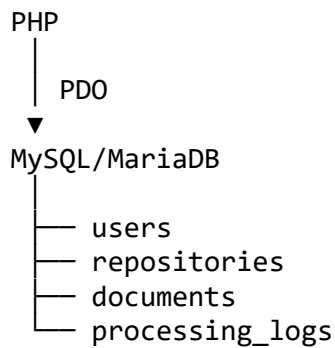
Seguridad

La arquitectura permite incorporar sesiones, CSRF, consultas preparadas y validaciones.

Integración IA

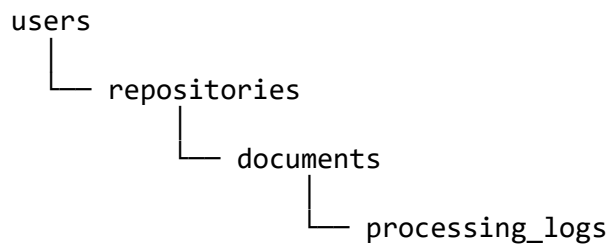
El procesador documental se mantiene separado de la lógica general de la aplicación, permitiendo cambiar o configurar el servicio de IA.

49. DISEÑO DE LA INTEGRACIÓN ENTRE BASE DE DATOS Y APLICACIÓN



Las relaciones permiten mantener la integridad de los datos.

Relación principal



50. DISEÑO DE ALMACENAMIENTO DOCUMENTAL

Se utilizan dos mecanismos complementarios.

50.1 Sistema de archivos

Conserva:

- PDF.
- DOCX.
- TXT.

50.2 Base de datos

Conserva:

- Metadatos.

- Contenido extraído.
- Categoría.
- Resumen.
- Datos extraídos.
- Estado.
- Errores.

Esta separación evita almacenar el archivo binario directamente dentro de la base de datos.

51. DISEÑO DE BÚSQUEDA

La tabla documents incorpora un índice FULLTEXT sobre:

```
original_name  
content  
summary
```

Esto permite diseñar búsquedas que no dependan únicamente del nombre del archivo, sino también del contenido procesado.

La existencia de este índice está definida directamente en el esquema SQL actual.

52. DISEÑO DE ESCALABILIDAD

Aunque la versión actual está orientada principalmente a un entorno académico/local, la arquitectura permite futuras mejoras.

Posibles extensiones:

- Servidor Linux.
- Base de datos independiente.
- Almacenamiento externo.
- Procesamiento asíncrono.
- Cola de trabajos.
- OCR.
- Embeddings.
- Base de datos vectorial.
- RAG.
- API REST.

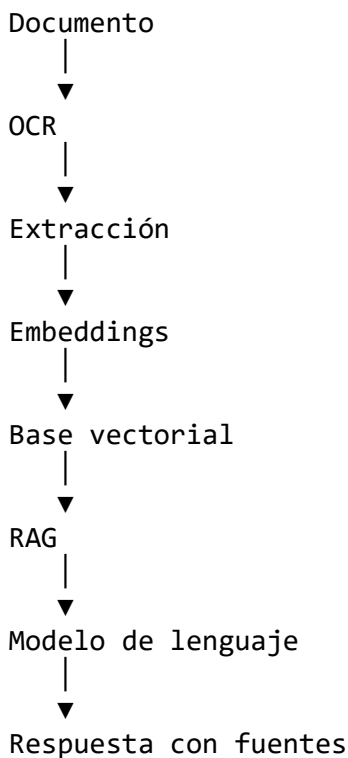
- Integración con servicios empresariales.

Estas características no forman parte de la versión actual y se consideran posibles líneas de evolución.

53. DISEÑO DE EVOLUCIÓN DE LA IA

La solución actual utiliza extracción textual y procesamiento basado en reglas junto con IA.

Una evolución futura podría ser:



Esta arquitectura no debe presentarse como parte de la implementación actual, sino como una posible evolución tecnológica.

54. REQUISITOS DE INFRAESTRUCTURA

54.1 Hardware mínimo de referencia

Para el entorno académico se recomienda:

- Procesador de 4 núcleos.
- 8 GB de RAM.
- 20 GB de espacio disponible.
- Conexión de red cuando se utilice un servicio IA externo.

Para ejecutar modelos locales mediante Ollama, los requerimientos dependerán del modelo utilizado.

55. SOFTWARE

Servidor

- Windows o sistema operativo compatible.
- Apache.
- PHP 8.x.
- MySQL/MariaDB.

Desarrollo

- Visual Studio Code u otro editor.
- Git.
- Navegador web.

IA

- Ollama, cuando se utilice procesamiento local.

Base de datos

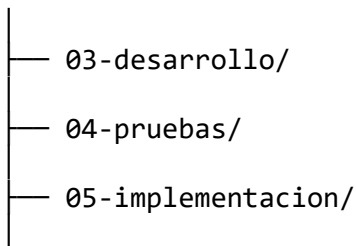
- MySQL/MariaDB.
 - phpMyAdmin como herramienta administrativa opcional.
-

56. ESTRUCTURA DEL PROYECTO

La estructura general del repositorio se organiza alrededor de las diferentes fases de ingeniería:

DesarrolloE2/

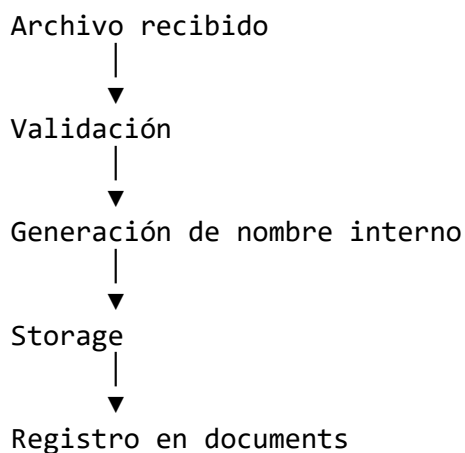
```
|  
├── 00-ingeneria-de-requerimientos/  
├── 01-analisis/  
└── 02-diseño/
```



Esta organización permite relacionar los entregables académicos con los componentes técnicos.

57. DISEÑO DE INTEGRACIÓN CON EL SISTEMA DE ARCHIVOS

El almacenamiento documental sigue el siguiente flujo:

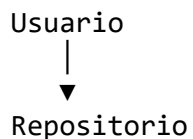


La base de datos conserva tanto el nombre original como el nombre utilizado internamente.

Esto permite mantener el nombre presentado al usuario sin utilizarlo necesariamente como nombre físico del archivo.

58. DISEÑO DE AUTORIZACIÓN

La autorización se basa principalmente en la relación:



↓
Documento

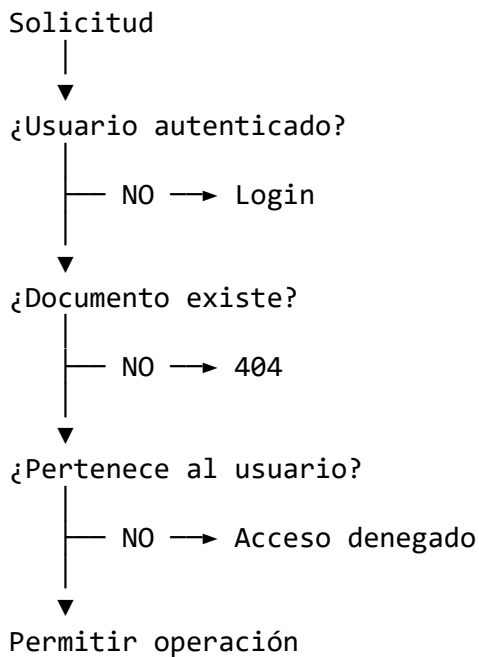
Para acceder a un documento:

1. Se identifica el usuario.
2. Se identifica el documento.
3. Se consulta el repositorio.
4. Se verifica que el repositorio pertenece al usuario.
5. Se permite la operación.

Este patrón se utiliza actualmente, por ejemplo, en la descarga de documentos.

59. DIAGRAMA DE FLUJO DE AUTORIZACIÓN

Flujo esperado



60. DISEÑO DE RESPUESTA DEL SISTEMA

Las respuestas del sistema deben clasificarse en:

Éxito

Ejemplo:

Documento procesado correctamente.

Advertencia

Ejemplo:

El documento fue cargado, pero no se pudo completar el análisis mediante IA.

Error

Ejemplo:

No fue posible procesar el documento.

Información

Ejemplo:

No se encontraron documentos relacionados con la consulta.

61. DISEÑO DE EXPERIENCIA DE USUARIO

El sistema debe mantener una navegación sencilla.

Principios:

- Acciones principales visibles.
 - Mensajes comprensibles.
 - Estados claramente identificados.
 - Resultados agrupados.
 - Formularios simples.
 - Confirmación de operaciones destructivas.
 - Información relevante priorizada.
-

62. DISEÑO DE OPERACIONES DESTRUCTIVAS

Las operaciones como eliminar documento o repositorio deberán solicitar confirmación.

Ejemplo:

¿Está seguro de que desea eliminar este documento?

Esto reduce el riesgo de eliminación accidental.

63. DISEÑO DE RESPALDO

Aunque el respaldo pertenece principalmente a la fase de implementación y administración, el diseño contempla que deben respaldarse dos tipos de información:

1. Base de datos.
2. Archivos originales.

Un respaldo de la base de datos sin los archivos físicos no permitiría recuperar completamente el repositorio documental.

64. DISEÑO DE RECUPERACIÓN

Ante una pérdida del sistema se deberá restaurar:

1. Aplicación
↓
2. Base de datos
↓
3. Directorio de documentos
↓
4. Configuración
↓
5. Servicio IA

La recuperación deberá verificarse posteriormente mediante pruebas.

65. MATRIZ DE RELACIÓN ENTRE COMPONENTES Y REQUISITOS

Componente	Requisitos relacionados
Login / sesión	RF-01, RF-02, RF-03
Repositorios	RF-04, RF-05, RF-06

Componente	Requisitos relacionados
Carga documental	RF-07, RF-08, RF-09
Procesador documental	RF-10, RF-11, RF-12, RF-13, RF-14
Servicio IA	RF-15
Base de datos	RF-16, RF-17, RF-18
Descarga	RF-19
Eliminación	RF-20
Búsqueda	RF-21
Consulta IA	RF-22, RF-23
Dashboard	RF-24

66. MATRIZ DE COMPONENTES Y TECNOLOGÍAS

Componente	Tecnología
Cliente	HTML, CSS, JavaScript
Servidor web	Apache
Backend	PHP 8
Persistencia	MySQL/MariaDB
Acceso BD	PDO
Sesiones	PHP Sessions
Documentos	Sistema de archivos
Extracción DOCX	ZipArchive/XML
Extracción PDF	Procesamiento implementado en PHP
IA	Ollama / API compatible
Control de versiones	Git
Entorno local	XAMPP

67. CONSIDERACIONES DE MANTENIBILIDAD

El diseño busca que los cambios futuros puedan realizarse de forma localizada.

Por ejemplo:

Cambiar el modelo IA

No debería requerir modificar toda la interfaz del sistema.

Agregar una nueva extensión

Se puede ampliar el procesador documental.

Agregar una categoría

Se pueden modificar las reglas de clasificación.

Agregar un indicador

Se puede ampliar la consulta utilizada por el dashboard.

Cambiar la base de datos

La capa de persistencia permite concentrar las operaciones mediante PDO.

68. LIMITACIONES DEL DISEÑO ACTUAL

Se reconocen las siguientes limitaciones:

1. El procesamiento PDF no reemplaza un sistema OCR.
2. No existe una base vectorial.
3. No se utilizan embeddings.
4. No se implementa RAG vectorial.
5. El procesamiento puede ser síncrono.
6. El entorno actual está orientado a ejecución local.
7. La escalabilidad horizontal no constituye un objetivo de esta versión.
8. El almacenamiento documental depende del sistema de archivos local.
9. La clasificación utiliza reglas y/o IA, no un modelo de clasificación entrenado específicamente para la organización.
10. La precisión de las respuestas depende de la calidad del contenido recuperado y del modelo configurado.

Estas limitaciones no representan necesariamente fallas del proyecto, sino decisiones relacionadas con el alcance académico establecido.

71. TRAZABILIDAD DISEÑO → REQUISITOS

Diseño	Requisitos
Arquitectura web	RF-01 a RF-24
Modelo de datos	RF-01, RF-04, RF-07, RF-10, RF-16, RF-17, RF-18

Diseño	Requisitos
Diseño documental	RF-07 a RF-18
Diseño IA	RF-11, RF-12, RF-13, RF-14, RF-15, RF-22
Diseño búsqueda	RF-21, RF-22, RF-23
Diseño seguridad	RF-01, RF-02, RF-03, RF-07, RF-19, RF-20
Diseño dashboard	RF-24
Diseño navegación	RF-01 a RF-24

72. TRAZABILIDAD DISEÑO → COMPONENTES

Requisito	Componente
RF-01	bootstrap.php, index.php, users
RF-04	index.php, repositories
RF-07	index.php, storage
RF-10	document_processor.php
RF-11	document_processor.php
RF-12	document_processor.php
RF-13	document_processor.php
RF-14	document_processor.php
RF-15	document_processor.php, Ollama/API
RF-16	MySQL/MariaDB
RF-17	documents
RF-18	processing_logs
RF-19	index.php, storage
RF-20	index.php, documents
RF-21	MySQL FULLTEXT
RF-22	PHP + IA
RF-23	PHP + documents
RF-24	Dashboard

73. CONCLUSIONES DEL DISEÑO

El diseño propuesto establece una estructura técnica coherente con el análisis realizado para DocuMind.

La arquitectura web permite separar las responsabilidades principales del sistema y mantener una comunicación clara entre interfaz, aplicación, procesamiento documental, persistencia y servicio de Inteligencia Artificial.

El modelo relacional utilizado proporciona una estructura adecuada para representar usuarios, repositorios, documentos y registros de procesamiento.

La separación entre archivos físicos y metadatos almacenados en la base de datos facilita la administración del repositorio documental.

El componente de procesamiento documental constituye una parte central de la solución, ya que permite convertir archivos PDF, DOCX y TXT en contenido susceptible de ser analizado.

La integración con IA se diseña como un componente complementario y configurable, permitiendo utilizar un servicio local como Ollama.

El diseño de seguridad contempla autenticación, sesiones, protección CSRF, consultas preparadas y validación de propiedad de los documentos.

Finalmente, los diagramas y mockups definidos como espacios dentro de este documento deberán ser incorporados como evidencia visual del diseño antes de presentar la versión definitiva.
